

Università degli Studi di Catania

Corso di Laurea in Informatica

Progetto di **Sistemi Cloud**

Mensa Universitaria

Applicazione a microservizi:
deployment su cluster Kubernetes locale e migrazione su AWS

Candidato
Vito Marino

Anno Accademico 2025/2026

Indice

1	Introduzione	5
1.1	Obiettivi del progetto	5
1.2	Requisiti	5
1.3	Struttura del documento	5
2	Architettura dell'applicazione	6
2.1	In cosa consiste l'applicazione	6
2.2	Suddivisione in microservizi	6
2.3	Persistenza dei dati	7
2.4	Flusso di un ordine	7
2.5	Configurazione esterna al codice	9
2.6	Containerizzazione	9
3	Fase 1: deployment su cluster Kubernetes locale	11
3.1	Obiettivo e struttura	11
3.2	Provisioning con Terraform e Multipass	11
3.3	Il collegamento tra Terraform e Ansible	12
3.4	Configurazione del cluster con Ansible	12
3.5	Manifest Kubernetes	13
3.6	Distribuzione delle immagini	13
3.7	Automazione della procedura di avvio	14
3.8	Verifica del deployment	14
4	Fase 2: migrazione su Amazon Web Services	15
4.1	Architettura di destinazione	15
4.2	Scelta tra EKS e cluster autogestito	16
4.3	Rete e sicurezza	16
4.4	Servizi gestiti	17
4.5	Gestione delle credenziali	17
4.6	Gestione dello stato di Terraform	18
4.7	Registro delle immagini	18
4.8	Bilanciamento del carico	18
4.9	Adattamenti al playbook Ansible	19
4.10	Pipeline ci/cd	19
4.10.1	Autenticazione senza credenziali statiche	19
4.10.2	Esecuzione del deployment	20
4.11	Automazione dell'avvio	20

5	Confronto, problemi affrontati e conclusioni	21
5.1	Corrispondenza tra le due fasi	21
5.2	Problemi affrontati	21
5.2.1	Concorrenza nella creazione dello schema	21
5.2.2	Assenza di ordinamento all'avvio in Kubernetes	22
5.2.3	Schema condiviso tra servizi	22
5.2.4	Vincoli dell'ambiente di sviluppo	22
5.2.5	Non determinismo nella selezione dell'immagine	23
5.2.6	Identificatori immutabili nel token OIDC	23
5.2.7	Visibilità delle immagini su S3	23
5.3	Considerazioni sui costi	24
5.4	Conclusioni	24

Capitolo 1

Introduzione

1.1 Obiettivi del progetto

Il progetto realizza una web app per la gestione degli ordini di una mensa universitaria e ne studia il deployment in due contesti differenti: un cluster Kubernetes installato su macchine virtuali locali e un'infrastruttura cloud basata su AWS.

L'obiettivo non è soltanto far funzionare l'applicazione, ma verificare una proprietà : ovvero che lo stesso identico codice applicativo possa essere eseguito in ambienti profondamente diversi senza modifiche, a patto che la configurazione sia esterna al codice. Questa proprietà viene dimostrata nella seconda fase, dove il passaggio dai container locali ai servizi gestiti di AWS avviene cambiando quasi esclusivamente variabili d'ambiente.

1.2 Requisiti

La richiesta della traccia consiste in:

- un'applicazione 3-tier basata su microservizi, con database relazionale ed eventuale comunicazione asincrona tramite code di messaggi;
- containerizzazione con Docker e orchestrazione con Kubernetes;
- un primo deployment su un cluster Kubernetes reale in locale, realizzato con Multipass;
- la gestione dell'infrastruttura tramite Infrastructure as Code, con Terraform e Ansible;
- la migrazione dell'architettura su AWS utilizzando servizi gestiti;
- una pipeline di integrazione CI/CD e distribuzione continua con GitHub Actions (nella versione Cloud).

1.3 Struttura del documento

Il documento è organizzato in due parti. Il Capitolo 2 descrive l'applicazione, che è comune a entrambe le fasi. Il Capitolo 3 tratta il deployment locale, il Capitolo 4 la migrazione su AWS. Il Capitolo 5 mette a confronto le due soluzioni e raccoglie i problemi incontrati durante lo sviluppo, che rappresentano una parte significativa del lavoro svolto. Le appendici riportano i listati principali.

Il codice è distribuito in due repository:

- <https://github.com/vitomarino02-del/Cloud-Mensa> (versione locale);
- <https://github.com/vitomarino02-del/Cloud-Mensa-AWS-Version> (versione AWS).

Capitolo 2

Architettura dell'applicazione

2.1 In cosa consiste l'applicazione

L'applicazione riproduce il funzionamento di una mensa universitaria. Uno studente consulta il menù del giorno, seleziona i piatti e invia un ordine; il personale di cucina visualizza gli ordini attivi su un tabellone e ne fa avanzare lo stato fino alla consegna. Ci sono 3 stati previsti per gli ordini: *ricevuto*, *in preparazione*, *pronto*,

2.2 Suddivisione in microservizi

L'applicazione è suddivisa in tre servizi backend e un frontend:

1. **menu-service:** Gestisce il menu, consente , attraverso routes API, di visualizzare i piatti disponibili (GET /api/menu), le relative foto (POST /api/menu/<id>/image) e con GET /Healthz controlla lo stato del servizio e permette a kubernetes di sapere se il servizio è vivo. La variabile STORAGE_BACKEND è importantissima ed è l'unico parametro che cambia tra la versione locale e quella cloud, infatti se siamo su cloud le immagini sono salvate su S3 (servizio amazon), mentre se siamo su Locale scrive su disco.
2. **order-service:** È il componente più importante. Riceve gli ordini e li lavora, mettendoli in coda e modificando il loro stato. Anche lui espone delle rotte API: con POST /api/orders vengono creati gli ordini; GET /api/orders?active=1 consente di visualizzare il tabellone della cucina (lo stesso utilizzato dal menu service), con GET /api/orders abbiamo lo storico e POST /api/orders/<id>/advance si avvanza lo stato. Come sopracitato, è l'unico a lavorare su tutti e 3 i sistemi di persistenza (PostgreSQL, Redis, RabbitMQ).
3. **kitchen-service:** Non espone nessuna API: è un semplice **consumatore**. Resta in ascolto sulla coda dei messaggi e reagisce agli eventi di creazione e di completamento degli ordini. Nella versione attuale registra le notifiche nei log; Dichiaro l'exchange order.events, crea la coda kitchen e la "consuma". Per ogni messaggio guarda il campo type e scrive nel log "nuovo ordine" o "ordine pronto". La comunicazione asincrona è dimostrata qui perché order-service pubblica e non sa né con chi sta parlando né aspetta la risposta, infatti attraverso il tipo fanout di exchange, come già detto, avviene la comunicazione.
4. **frontend:** Pagina singola in React, compilata e servita da Nginx. Nginx svolge anche la funzione di gateway applicativo, instradando le richieste /api/menu e /api/orders

verso i rispettivi servizi. Questa scelta fa sì che l'unico componente esposto verso l'esterno sia il frontend: i microservizi e i sistemi di persistenza non sono mai raggiungibili direttamente. Come conseguenza del fatto che Nginx instrada il traffico utilizzando il livello 7 vi è la scelta progettuale di utilizzare il NLB anziché l'ALB, infatti quest'ultimo già lavora nel livello 7 e sarebbe un componente con un costo maggiore non utilizzato.

2.3 Persistenza dei dati

Il progetto utilizza tre sistemi di persistenza distinti, ciascuno con un ruolo preciso.

PostgreSQL è il database che contiene i piatti, lo storico degli ordini e le relative righe, con vincoli di integrità e transazioni. La struttura prevede tre tabelle: `dishes`, `orders` e `order_items`.

Un dettaglio progettuale riguarda, ad esempio, `order_items`: il nome e il prezzo unitario del piatto, sono infatti *copiati* solo nel momento dell'ordine invece di essere referenziati. In questa maniera se venisse cambiato un prezzo non verrebbero influenzati anche gli ordini precedenti. Non è pensato per essere interrogato più volte al secondo dal display della cucina ed è il motivo per cui è stato scelto l'utilizzo di Redis.

Redis è un archivio chiave-valore in memoria ed è molto più veloce negli accessi di un database relazionale (a differenza di Postgres o RDS), poiché il display cucina fa polling molto frequentemente, ci sarebbe un'operazione costosa ripetuta continuamente che Redis fa, invece, con una singola operazione in memoria. A tal proposito vi è un unico hash chiamato `kitchen:board`, dove il campo è l'id dell'ordine e il valore ' il json già serializzato così da consentire la risposta pronta da restituire al frontend e non richiedere nessuna ricomposizione. Le operazioni utilizzate sono HSET per inserire (o aggiornare) un ordine e HDEL per toglierlo.

Bisogna sottolineare che Redis non contiene dati critici: tutto ciò che è memorizzato qui è ricostruibile da PostgreSQL. Coerentemente, il codice tratta l'indisponibilità di Redis come una condizione di degrado e non di errore, ogni operazione è realizzata con try/except e in caso di errore viene inviato un warning e si continua.

RabbitMQ è un message broker, un intermediario che inoltra messaggi da chi produce a chi li consuma, attraverso comunicazione asincrona. Quando un ordine viene creato, order-service pubblica un evento su un exchange di tipo *fanout* chiamato `order.events`. La scelta del fanout è deliberata: il produttore non conosce i consumatori, e un eventuale nuovo servizio (statistiche, notifiche push, integrazione contabile) può iscriversi all'exchange senza che il codice di order-service venga modificato.

2.4 Flusso di un ordine

Il percorso completo di un ordine attraversa tutti i componenti descritti:

1. il browser invia POST `/api/orders` a Nginx;
2. Nginx, in base al path, inoltra la richiesta a order-service;

```
1 location /api/menu { proxy_pass http://menu-service:5001; }
2 location /api/orders { proxy_pass http://order-service:5002; }
3
4 location / { try_files $uri $uri/ /index.html; }
5 }
```

Listing 2.1: righe 9-12 di locale/frontend/nginx.conf

Nginx qui fa da gateway e fa routing di livello 7, utilizza menu-service e order-service che sono i nomi dei service Kubernetes, risolti dal DNS interno del cluster.

3. **order-service verifica l'esistenza dei piatti e calcola il totale *lato server*, senza fidarsi dei valori inviati dal client;**

```

1 if not customer or not items: return {"error": "cliente e items
  obbligatori"}, 400 #verifica 1
2 dish = db.session.get(Dish, it["dish_id"])
3 if not dish: return {"error": "piatto inesistente"}, 400 #verifica 2

```

Listing 2.2: validazione in order-service/app.py, righe 143-159

Una volta trovato il piatto, il server non utilizza il prezzo inviato dal client, ma prende il prezzo direttamente dal database:

```

1 total += float(dish.price) * qty

```

Il principio è che il server non si fida mai dell'input. Se il totale arrivasse dalla richiesta, chiunque potrebbe modificarlo con gli strumenti di sviluppo del browser e ordinare a un euro.

4. **l'ordine viene salvato su PostgreSQL con un commit transazionale;**

```

1 order = Order(customer=customer, status="ricevuto", total=0) # 148
2 db.session.add(order) #149
3 ...
4 order.items.append(OrderItem(...)) #157 (nel ciclo)
5 ...
6 db.session.commit() #160

```

5. **l'ordine viene inserito nel tabellone Redis;**

```

1 board_put(order.to_dict()) # 161
2
3 def board_put(order): # 81
4     try:
5         r().hset(BOARD_KEY, order["id"], json.dumps(order)) # 83
6     except Exception as e:
7         log.warning("Redis non disponibile: %s", e) # 84-85

```

Listing 2.3: riga 161 che richiama la funzione alle righe 81-86

Il try/except consente come già detto che in caso di errore il display non visualizza l'ordine ma lo salva comunque.

6. **viene pubblicato l'evento order.created su RabbitMQ;**

```

1     publish("order.created", order.to_dict()) # 162
2
3 def publish(event_type, payload): # 105
4     body = json.dumps({"type": event_type, "data": payload}) # 106
5     try:
6         conn =
7             pika.BlockingConnection(pika.URLParameters(RABBITMQ_URL))
8             # 108
9         ch = conn.channel() # 109
10        ch.exchange_declare(exchange=EXCHANGE, exchange_type="fanout",
11                             durable=True) # 110
12        ch.basic_publish(exchange=EXCHANGE, routing_key="", body=body)
13        # 111
14        conn.close() # 112
15    except Exception as e:
16        log.warning("RabbitMQ non disponibile: %s", e) #
17        113-114

```


Listing 2.4: riga 162, funzioni 105-114

La routing key="" alla riga 111 è la conseguenza del fanout: la chiave non viene guardata. E in tutto il file non compare mai il nome di una coda. Questa assenza fa sì che l'affermazione "il produttore non conosce i consumatori" sia vera.

7. **kitchen-service, in ascolto, riceve l'evento e notifica la cucina;** (in parallelo con p.8)

```

1  ch.exchange_declare(exchange=EXCHANGE, exchange_type="fanout",
    durable=True) # 40-41
2  q = ch.queue_declare(queue="kitchen", durable=True)
    # 42
3  ch.queue_bind(exchange=EXCHANGE, queue=q.method.queue)
    # 43
4  ch.basic_consume(queue=q.method.queue, on_message_callback=on_message,
    auto_ack=True) # 45-46
5  ch.start_consuming()
```

Listing 2.5: kitchen-service/consumer.py, righe 40-47:

8. **il display di cucina, interrogando GET /api/orders?active=1, legge il tabellone da Redis e mostra il nuovo ordine.** (in parallelo con p.7)

L'ordine delle operazioni 4, 5 e 6 non è casuale: la scrittura sul database avviene per prima perché è l'unica che non può essere persa. Le due successive interessano sistemi ausiliari e sono gestite in modo che un loro fallimento non comprometta l'ordine già registrato.

2.5 Configurazione esterna al codice

Ogni servizio legge la propria configurazione da variabili d'ambiente:

```

1  STORAGE_BACKEND = os.environ.get("STORAGE_BACKEND", "local")
2  IMAGE_DIR = os.environ.get("IMAGE_DIR", "/data/images")
3  app.config["SQLALCHEMY_DATABASE_URI"] = os.environ.get(
4  "DATABASE_URL", "postgresql://mensa:mensa@localhost:5432/mensa")
```

Listing 2.6: Lettura della configurazione in menu-service

Questa impostazione, apparentemente marginale, è ciò che rende possibile l'intera seconda fase del progetto: gli stessi container che in locale si collegano ai pod di PostgreSQL, Redis e RabbitMQ, su AWS si collegano a RDS, ElastiCache e Amazon MQ senza alcuna modifica al codice.

Lo stesso principio governa la gestione delle immagini dei piatti. Il menu-service espone due funzioni, `save_image` e `image_url`, il cui comportamento dipende dalla variabile `STORAGE_BACKEND`: con il valore `local` i file vengono scritti su un volume, con il valore `s3` vengono caricati su un bucket Amazon S3 tramite la libreria `boto3`.

2.6 Containerizzazione

Ogni servizio è distribuito come immagine Docker autonoma, contenente il proprio codice, le proprie dipendenze e le istruzioni di avvio. La struttura di ciascuna directory di servizio è volutamente identica: il file applicativo, `requirements.txt` con le versioni bloccate e il `Dockerfile`. Le immagini di PostgreSQL, Redis e RabbitMQ sono quelle ufficiali, prelevate da Docker Hub: nel `docker-compose.yml` compaiono con la direttiva `image:`, mentre nei manifest Kubernetes sono scaricate direttamente dal containerd di ciascun nodo.

Le immagini dei servizi applicativi derivano a loro volta da immagini base di Docker Hub (`python:3.12-slim`, `node:20-alpine`, `nginx:1.27-alpine`) e sono costruite localmente dai rispettivi `Dockerfile`, per essere poi pubblicate sullo stesso registro e prelevate dai nodi al momento dell'esecuzione. Il meccanismo di distribuzione è quindi il medesimo per tutte le immagini impiegate, ed è lo stesso che nella Fase 2 viene realizzato con Amazon ECR.

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 EXPOSE 5001
7 CMD ["gunicorn", "-b", "0.0.0.0:5001", "-w", "2", "app:app"]
```

Listing 2.7: Dockerfile di menu-service

Capitolo 3

Fase 1: deployment su cluster Kubernetes locale

3.1 Obiettivo e struttura

La prima fase (locale) consiste nel deployment dell'applicazione su un cluster Kubernetes reale, installato su macchine virtuali locali. L'infrastruttura è descritta interamente come codice: Terraform crea le macchine virtuali, Ansible le configura trasformandole in un cluster, i manifest Kubernetes descrivono l'applicazione.

Al fianco di questo percorso ne esiste anche un altro, più breve, pensato solo ed esclusivamente per test rapidi in sviluppo. Avviare il cluster richiede la creazione di tre macchine virtuali e la loro configurazione completa, operazione che richiede diversi minuti: questo tempo è accettabile per il deployment, ma non per fare dei piccoli test o modifiche e riavviare frequentemente il tutto. Per questo motivo è stato aggiunto anche un `docker-compose.yml` che avvia tutta l'applicazione direttamente sul PC e senza macchina virtuale.

Le due modalità utilizzano gli stessi Dockerfile e le stesse immagini ma differiscono per l'orchestratore e per il modo in cui viene fornita la configurazione, dichiarata nel file di Compose in un caso e in una ConfigMap nell'altro. Docker Compose si limita ad avviare i container su un singolo host; Kubernetes introduce invece la distribuzione su più nodi, le repliche e il ripristino automatico dei pod, ed è quindi il deployment di riferimento di questa fase.

3.2 Provisioning con Terraform e Multipass

Terraform è utilizzato per creare tre macchine virtuali Ubuntu tramite Multipass: una destinata al control plane e due ai nodi worker. Le risorse sono state descritte utilizzando l'approccio dichiarativo:

```
1 resource "multipass_instance" "worker" {
2   count          = var.worker_count
3   name           = "mensa-worker-${count.index + 1}"
4   image          = "22.04"
5   cpus           = var.cpus
6   memory         = var.memory
7   disk           = var.disk
8   cloudinit_file = local_file.cloudinit.filename
9 }
```

Listing 3.1: Codice di definizione dei nodi worker

Le specifiche sono state scelte per via di alcuni test e sono i requisiti minimi verificati da kubeadm durante i controlli preliminari.

Al primo avvio ciascuna macchina esegue uno script *cloud-init* che crea l'utente di servizio e vi installa la chiave SSH pubblica dell'operatore. Questo passaggio è ciò che consente ad Ansible di collegarsi successivamente senza password.

3.3 Il collegamento tra Terraform e Ansible

Gli indirizzi IP assegnati alle macchine virtuali non sono noti prima della loro creazione. Terraform li legge al termine del provisioning e genera automaticamente il file di inventario di Ansible:

```

1 resource "local_file" "ansible_inventory" {
2   filename = "${path.module}/ansible/inventory.ini"
3   content = templatefile("${path.module}/inventory.tpl", {
4     cp_ip      = multipass_instance.control_plane.ipv4
5     worker_ips = [for w in multipass_instance.worker : w.ipv4]
6     ssh_key_path = pathexpand(var.ssh_private_key_path)
7   })
8 }
```

Listing 3.2: Generazione del file inventory.ini

Il risultato è una catena: Terraform crea le macchine, cloud-init installa la chiave SSH, Terraform scrive l'inventario, Ansible successivamente lo legge e si collega.

3.4 Configurazione del cluster con Ansible

Il playbook `site.yml` è organizzato in tre *play* corrispondenti alle tre fasi logiche dell'installazione.

Il primo play riguarda tutti i nodi e ne prepara il sistema operativo: disattiva la memoria di swap (rifiutata da kubelet), carica i moduli del kernel `overlay` e `br_netfilter`, imposta i parametri di rete necessari all'instradamento tra pod, installa il runtime dei container (containerd) e i pacchetti `kubelet`, `kubeadm` e `kubect1`.

È opportuno soffermarsi sulla scelta del runtime. Sui nodi non viene installato Docker ma **containerd**. Kubernetes non dialoga direttamente con Docker: utilizza l'interfaccia standard CRI, e containerd la implementa nativamente. Docker resta necessario soltanto sulla macchina di sviluppo, dove le immagini vengono costruite.

Il secondo play inizializza il control plane con `kubeadm init`, installa la rete dei pod (Flannel), genera il comando di join e scarica il file `kubeconfig`. Il terzo play esegue il join sui nodi worker.

```

1 - name: kubeadm init
2   command: >
3     kubeadm init
4     --pod-network-cidr=10.244.0.0/16
5     --apiserver-advertise-address={{ ansible_host }}
6   args:
7     creates: /etc/kubernetes/admin.conf
```

Listing 3.3: Inizializzazione del control plane

L'attributo `creates` merita attenzione: `kubeadm init` non è un comando idempotente, e rieseguirlo su un cluster già inizializzato produrrebbe un errore. Indicando un file che il comando genera, Ansible verifica la presenza di tale file e, se esiste, salta l'operazione. L'intero

playbook può quindi essere rieseguito senza effetti collaterali, che è la proprietà fondamentale degli strumenti di configuration management.

3.5 Manifest Kubernetes

I manifest descrivono lo stato desiderato dell'applicazione. Per ciascun servizio sono definiti un **Deployment**, che mantiene attivo un numero prestabilito di repliche, e un **Service**, che fornisce un nome DNS stabile e distribuisce il traffico tra le repliche disponibili.

La distinzione tra i due oggetti è essenziale: i pod sono entità effimere, con indirizzi IP che cambiano a ogni ricreazione; il Service costituisce il punto di accesso stabile. Se un pod termina, il Deployment ne crea immediatamente un altro e il Service smette di indirizzargli traffico: è il meccanismo di *self-healing* che distingue Kubernetes da un semplice orchestratore di container.

Il servizio order-service è configurato con due repliche, a dimostrazione della scalabilità orizzontale; kitchen-service, non ricevendo richieste, è definito senza Service associato; il frontend è esposto tramite un Service di tipo **NodePort** sulla porta 30080.

```
1 spec:
2   replicas: 2
3   template:
4     spec:
5       containers:
6         - name: order-service
7           image: mensa/order-service:1.0
8           imagePullPolicy: Never
9           envFrom:
10            - configMapRef: { name: mensa-config }
11           readinessProbe:
12             httpGet: { path: /healthz, port: 5002 }
```

Listing 3.4: Estratto del Deployment di order-service

La direttiva `imagePullPolicy: Never` è legata all'assenza di un registro di immagini in ambiente locale, questione affrontata nel paragrafo seguente.

3.6 Distribuzione delle immagini

Le immagini costruite sulla macchina di sviluppo devono essere accessibili ai nodi del cluster, che dispongono di un proprio archivio gestito da containerd e non hanno visibilità sul daemon Docker della macchina di sviluppo. La soluzione adottata è la pubblicazione su un registro: lo script `push-images.sh` costruisce le quattro immagini e le carica su Docker Hub, dal quale ciascun nodo le preleva autonomamente quando lo scheduler vi assegna il pod corrispondente.

```
1 for s in $SERVICES; do
2   docker build -t "$DOCKER_USER/$s:$TAG" ./$s
3   docker push "$DOCKER_USER/$s:$TAG"
4 done
```

Listing 3.5: Pubblicazione delle immagini sul registro

Nei manifest la direttiva `imagePullPolicy: IfNotPresent` indica di prelevare l'immagine dal registro soltanto quando non è già presente nell'archivio locale del nodo: con un tag fisso ciò evita trasferimenti ripetuti a ogni riavvio dei pod. Il medesimo meccanismo governa già le immagini ufficiali di PostgreSQL, Redis e RabbitMQ, anch'esse prelevate da Docker Hub.

L'adozione di un registro rende l'aggiunta di un nodo al cluster un'operazione priva di passaggi aggiuntivi e costituisce lo stesso modello impiegato nella Fase 2, dove Docker Hub è sostituito da Amazon ECR. Nella prima versione del progetto era stata pensata una soluzione alternativa che consisteva nel caricamento dei container direttamente sulle macchine virtuali.

3.7 Automazione della procedura di avvio

I passaggi descritti nelle sezioni precedenti sono numerosi e devono essere eseguiti nell'ordine corretto: la creazione delle macchine, la configurazione del cluster, il caricamento delle immagini nei nodi e infine l'applicazione dei manifest. Per rendere la procedura ripetibile e meno soggetta a errori, l'intera sequenza è incapsulata nello script `up.sh`, che richiama in ordine gli strumenti già descritti e riporta l'avanzamento a ciascun passaggio.

Lo script non riscrive alcuna logica: si limita a orchestrare i componenti esistenti, che restano utilizzabili singolarmente. È così possibile, ad esempio, ricostruire e ridistribuire le sole immagini dopo una modifica al codice, senza intervenire sull'infrastruttura.

L'unica operazione aggiuntiva rispetto alla sequenza manuale è un controllo preliminare di connettività, introdotto in seguito al problema di rete riscontrato.

```
1 if ! multipass exec mensa-cp -- ping -c1 -W3 8.8.8.8 > /dev/null 2>&1; then
2   echo "le VM non escono su internet, applico le regole di rete"
3   sudo bash fix-rete-wsl.sh
4   sleep 3
5   if ! multipass exec mensa-cp -- ping -c1 -W3 8.8.8.8 > /dev/null 2>&1; then
6     echo "ERRORE: le VM continuano a non raggiungere internet."
7     exit 1
8   fi
9 fi
```

Listing 3.6: Verifica della connettività prima di configurare il cluster

Il controllo viene eseguito prima di avviare il playbook, poiché senza accesso alla rete l'installazione dei pacchetti fallirebbe a metà configurazione, lasciando il cluster in uno stato incompleto. Se le macchine non raggiungono internet lo script applica le regole necessarie e ripete la verifica, interrompendosi soltanto se il problema persiste. Si tratta di un esempio del principio generale adottato in tutto il progetto: rendere esplicite le precondizioni anziché darle per soddisfatte.

Lo script `down.sh` esegue l'operazione inversa, distruggendo le macchine virtuali. Per una sospensione temporanea è preferibile `multipass stop -all`, che le spegne senza eliminarle: al successivo avvio il cluster torna operativo autonomamente, poiché kubelet è configurato per partire con il sistema.

3.8 Verifica del deployment

Al termine della procedura il cluster risulta composto da tre nodi in stato **Ready** e i pod dell'applicazione in stato **Running**. L'applicazione risponde sulla porta 30080 di ciascun nodo.

La verifica funzionale consiste nell'inserire un ordine dall'interfaccia, osservare la comparsa dell'evento nei log di kitchen-service (a conferma del funzionamento della coda) e farne avanzare lo stato dal display di cucina fino alla rimozione dal tabellone Redis.

Capitolo 4

Fase 2: migrazione su Amazon Web Services

4.1 Architettura di destinazione

La seconda fase riproduce la stessa applicazione su AWS, sostituendo i componenti di supporto con i corrispondenti servizi gestiti e mantenendo Kubernetes come piattaforma di orchestrazione.

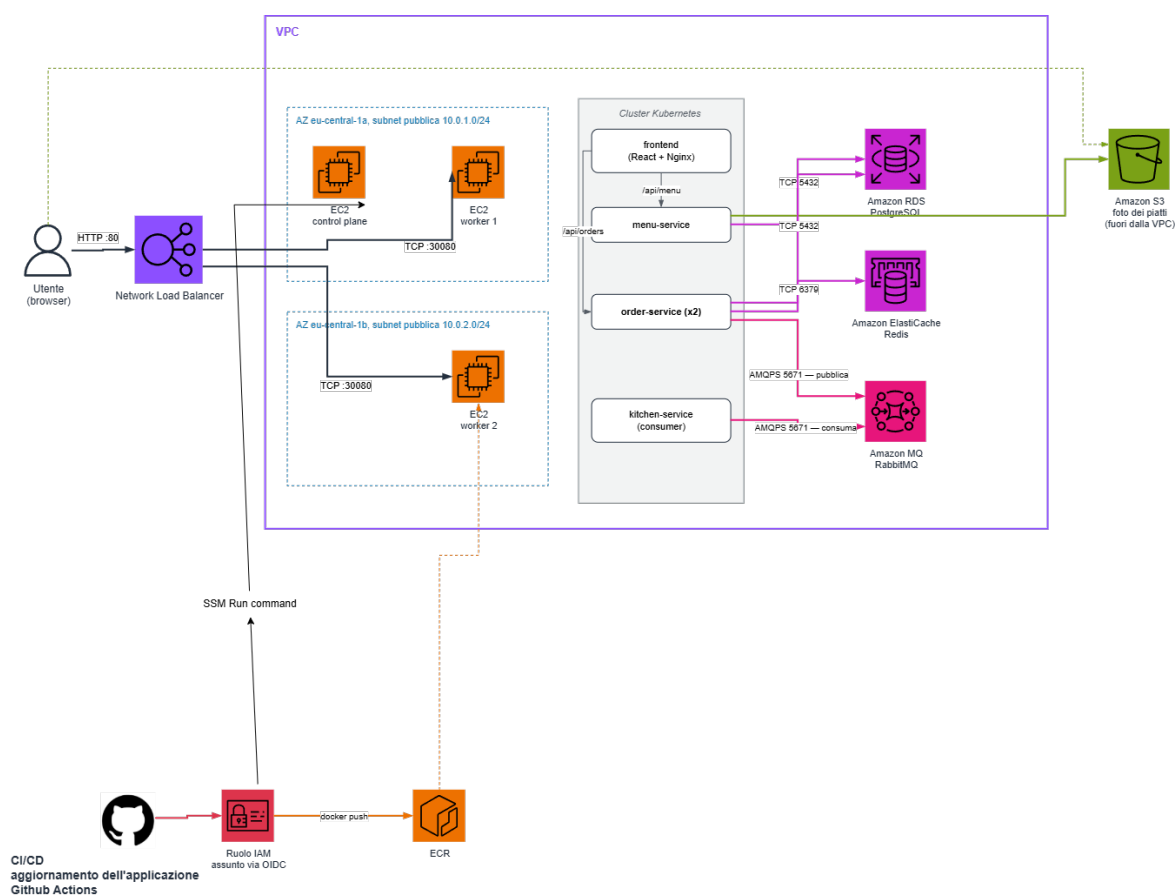


Figura 4.1: Architettura dell'applicazione su AWS

L'architettura, illustrata in Figura 4.1, comprende una VPC dedicata che ospita il cluster

Kubernetes su tre istanze EC2, un Network Load Balancer come unico punto di ingresso pubblico, i servizi gestiti per la persistenza e la messaggistica, un registro privato per le immagini e i componenti di supporto all'automazione.

4.2 Scelta tra EKS e cluster autogestito

La traccia consentiva sia l'utilizzo di Amazon EKS sia l'installazione di un cluster Kubernetes su istanze EC2. È stata scelta la seconda opzione per tre motivi.

Il primo è economico: EKS comporta un costo fisso per il control plane indipendente dall'utilizzo effettivo, difficilmente giustificabile in un progetto didattico che viene acceso e spento ripetutamente. Il secondo è di continuità: il playbook Ansible sviluppato nella prima fase è riutilizzabile quasi integralmente, il che rende evidente il vantaggio di aver descritto la configurazione in modo dichiarativo e indipendente dall'infrastruttura sottostante. Il terzo è didattico: gestire direttamente il control plane comporta la comprensione di aspetti che EKS nasconde, come la pubblicazione dell'endpoint dell'API server e la gestione dei certificati.

Il contro di questa scelta va riconosciuto: il control plane è singolo e non replicato, e la sua manutenzione (aggiornamenti, backup di etcd) resta a carico dell'operatore. In un contesto di produzione la scelta sarebbe diversa.

4.3 Rete e sicurezza

La rete è costituita da una VPC con blocco di indirizzi 10.0.0.0/16 e due sottoreti pubbliche collocate in due Availability Zone distinte. La distribuzione su due zone non è opzionale: RDS ed ElastiCache richiedono un gruppo di sottoreti su almeno due zone, e costituisce inoltre il presupposto per l'alta disponibilità.

Si è scelto di non utilizzare un NAT Gateway. Questo componente comporterebbe un costo fisso mensile non trascurabile, e la sua funzione (consentire il traffico in uscita a istanze prive di indirizzo pubblico) non è necessaria in questo contesto: i nodi risiedono in sottoreti pubbliche e la loro protezione è affidata ai security group.

Sono definiti due security group con ruoli distinti. Il primo protegge i nodi del cluster: consente SSH e accesso all'API server esclusivamente dall'indirizzo IP dell'operatore, la porta 30080 per il traffico proveniente dal load balancer, e il traffico libero tra i nodi stessi, necessario alla comunicazione interna del cluster.

```
1 ingress {
2   description = "Traffico tra i nodi"
3   from_port   = 0
4   to_port     = 0
5   protocol    = "-1"
6   self        = true
7 }
```

Listing 4.1: Regola per il traffico interno al cluster

Il secondo security group protegge i servizi gestiti e presenta una caratteristica interessante: le regole non fanno riferimento a intervalli di indirizzi ma al security group dei nodi.

```
1 ingress {
2   description      = "PostgreSQL"
3   from_port        = 5432
4   to_port          = 5432
5   protocol         = "tcp"
6   security_groups  = [aws_security_group.nodes.id]
```



```
7 }
```

Listing 4.2: Accesso a PostgreSQL limitato ai soli nodi

Il controllo avviene quindi per *identità* e non per indirizzo: qualunque istanza appartenga al gruppo dei nodi può raggiungere il database, e nessun'altra. Poiché gli indirizzi delle istanze cambiano a ogni ricreazione, questa forma di regola è anche l'unica praticabile.

4.4 Servizi gestiti

Amazon RDS sostituisce il pod PostgreSQL. L'istanza è configurata con cifratura dello storage e, soprattutto, con `publicly_accessible` impostato a falso: il database non possiede un indirizzo pubblico ed è raggiungibile unicamente dall'interno della VPC.

Amazon ElastiCache sostituisce il pod Redis. La configurazione è volutamente minimale, senza repliche né backup: coerentemente con il ruolo assegnato a Redis nell'architettura, i dati contenuti sono ricostruibili.

Amazon MQ sostituisce il pod RabbitMQ. Questo componente ha richiesto due adattamenti. Il primo riguarda il dimensionamento: AWS non supporta più la classe `mq.t3.micro` con il motore RabbitMQ, riservandola ad ActiveMQ; è stata quindi utilizzata `mq.m7g.medium`, la classe più economica disponibile. Il secondo riguarda il protocollo: Amazon MQ espone esclusivamente AMQPS, ovvero AMQP incapsulato in TLS, sulla porta 5671 anziché la 5672 in chiaro. La libreria `pika` gestisce autonomamente gli URI con schema `amqps://`, per cui l'adattamento si è limitato alla stringa di connessione e all'apertura della porta corretta nel security group.

Amazon S3 sostituisce il volume delle immagini. È il punto in cui l'astrazione predisposta nella prima fase produce il suo effetto: impostando `STORAGE_BACKEND=s3` il ramo `boto3` del codice, fino a quel momento inutilizzato, inizia a scrivere sul bucket. A differenza del volume locale, il bucket è persistente e condiviso tra tutte le repliche del servizio.

4.5 Gestione delle credenziali

Nessuna credenziale è presente nei repository. Le password vengono generate da Terraform in fase di creazione:

```
1 resource "random_password" "db" {
2   length = 20
3   special = false
4 }
```

Listing 4.3: Generazione della password del database

L'esclusione dei caratteri speciali evita problemi di interpretazione all'interno degli URI di connessione. Terraform compone quindi le stringhe di connessione complete, utilizzando le password generate e gli endpoint assegnati da AWS, e le memorizza cifrate su **SSM Parameter Store**. Lo script di deployment le rilegge da questo archivio e le trasforma in un Secret di Kubernetes.

La catena completa è quindi:

1. Terraform genera e scrive su SSM

2. lo script legge da SSM e crea il Secret,
3. Kubernetes inietta il Secret come variabili d'ambiente nel pod.

In nessun punto della catena una credenziale transita per il repository.

Un discorso simile riguarda l'accesso ad S3. Alle istanze EC2 è associato un ruolo IAM con i permessi minimi necessari: lettura dal registro delle immagini e scrittura limitata al solo bucket delle foto. La libreria boto3 in esecuzione nel pod ottiene credenziali temporanee dal servizio di metadati dell'istanza, senza che alcuna chiave sia mai configurata.

4.6 Gestione dello stato di Terraform

Nella seconda fase lo stato di Terraform non risiede più sulla macchina locale ma su un bucket S3, con una tabella DynamoDB utilizzata come meccanismo di lock:

```

1 backend "s3" {
2   bucket      = "mensa-tfstate-<account-id>"
3   key         = "mensa/aws/terraform.tfstate"
4   region      = "eu-central-1"
5   dynamodb_table = "mensa-tfstate-lock"
6   encrypt     = true
7 }

```

Listing 4.4: Configurazione del backend remoto

Le motivazioni sono tre. Lo stato deve essere accessibile sia dalla postazione di sviluppo sia dalla pipeline automatica; deve essere versionato, perché la sua perdita comporterebbe l'impossibilità per Terraform di riconoscere le risorse già create, che resterebbero attive e fatturate; deve infine essere protetto da accessi concorrenti, poiché due esecuzioni simultanee di `apply` produrrebbero uno stato incoerente.

Il bucket e la tabella costituiscono un problema di circolarità: Terraform non può creare le risorse in cui salva il proprio stato. Sono quindi creati una sola volta tramite AWS CLI, operazione documentata nel README del progetto.

4.7 Registro delle immagini

Amazon ECR ospita le quattro immagini dell'applicazione. La definizione utilizza il costrutto `for_each` per evitare ripetizioni:

```

1 resource "aws_ecr_repository" "services" {
2   for_each = toset(["menu-service", "order-service",
3                     "kitchen-service", "frontend"])
4   name      = "${var.project}/${each.key}"
5   force_delete = true
6 }

```

Listing 4.5: Creazione dei repository ECR

L'introduzione del registro elimina lo script di trasferimento manuale utilizzato nella prima fase: i nodi scaricano le immagini autonomamente, autorizzati dal ruolo IAM loro associato.

4.8 Bilanciamento del carico

Un Network Load Balancer distribuisce il traffico verso la porta 30080 dei nodi worker. La scelta di un bilanciatore di livello 4 anziché di livello 7 è motivata dalla semplicità del caso

d'uso: non essendo necessario alcun instradamento basato su percorso o intestazioni HTTP, un Application Load Balancer introdurrebbe complessità e costi senza benefici.

4.9 Adattamenti al playbook Ansible

Il playbook della prima fase è stato riutilizzato con tre modifiche, tutte conseguenza del fatto che il cluster è ora raggiungibile da Internet.

L'API server pubblicizza l'indirizzo *privato*, poiché la comunicazione tra i nodi avviene all'interno della VPC. Tuttavia, per poter utilizzare `kubectl` dalla postazione di sviluppo, il certificato deve includere anche l'indirizzo *pubblico*, altrimenti la connessione verrebbe rifiutata per non corrispondenza del nome:

```

1 - name: kubeadm init
2   command: >
3     kubeadm init
4     --pod-network-cidr=10.244.0.0/16
5     --apiserver-advertise-address={{ ansible_default_ipv4.address }}
6     --apiserver-cert-extra-sans={{ ansible_host }}
```

Listing 4.6: Inizializzazione con doppio indirizzo

Infine, il file `kubeconfig` scaricato contiene l'indirizzo privato e viene riscritto localmente con quello pubblico.

4.10 Pipeline ci/cd

La pipeline, realizzata con GitHub Actions, si attiva a ogni push sul ramo principale ed esegue quattro operazioni: autenticazione su AWS, costruzione delle immagini, caricamento su ECR e aggiornamento dei Deployment.

4.10.1 Autenticazione senza credenziali statiche

L'autenticazione utilizza OpenID Connect (OIDC). GitHub emette un token firmato che attesta l'identità del workflow; AWS lo verifica e restituisce credenziali temporanee valide un'ora. Nessuna chiave di accesso permanente viene quindi memorizzata nei secret del repository.

La fiducia è stabilita tramite un identity provider e un ruolo IAM la cui policy di assunzione verifica il campo `sub` del token:

```

1 Condition = {
2   StringEquals = {
3     "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
4   }
5   StringLike = {
6     "token.actions.githubusercontent.com:sub" = [
7       "repo:${var.github_repo}:*",
8       "repo:vitomarin02-del@*/Cloud-Mensa-AWS-Version@*:*"
9     ]
10  }
11 }
```

Listing 4.7: Policy di assunzione del ruolo

La presenza di due modelli alternativi è la conseguenza di un problema affrontato durante lo sviluppo, descritto nel Paragrafo 5.2.6.

4.10.2 Esecuzione del deployment

L'aggiornamento del cluster presenta un ostacolo: l'API server accetta connessioni soltanto dall'indirizzo dell'operatore, mentre i runner di GitHub dispongono di indirizzi variabili e non prevedibili. Aprire l'API server al traffico proveniente da qualunque origine sarebbe stata la soluzione più immediata, ma avrebbe indebolito sensibilmente la sicurezza.

Si è scelto invece di utilizzare **SSM Run Command**: la pipeline non si collega al cluster, ma richiede ad AWS di eseguire un comando *all'interno* del control plane, dove **kubectl** opera in locale. Non viene aperta alcuna porta aggiuntiva e non circolano credenziali del cluster.

Le immagini sono etichettate con l'identificativo del commit anziché con una versione fissa. Questa scelta garantisce la tracciabilità: osservando i Deployment attivi è sempre possibile risalire esattamente alla revisione del codice in esecuzione.

4.11 Automazione dell'avvio

L'intera procedura di avvio è incapsulata nello script **up.sh**, che richiama in sequenza gli strumenti già descritti: rileva l'indirizzo IP corrente, crea l'infrastruttura, configura il cluster, costruisce e carica le immagini, distribuisce l'applicazione e ne popola il catalogo di immagini. Lo script **down.sh** esegue l'operazione di distruzione.

Si tratta di un orchestratore e non di una riscrittura: i singoli script restano utilizzabili autonomamente, il che consente ad esempio di aggiornare l'applicazione senza toccare l'infrastruttura.

Capitolo 5

Confronto, problemi affrontati e conclusioni

5.1 Corrispondenza tra le due fasi

La Tabella 5.1 riassume le corrispondenze tra i componenti delle due fasi.

Tabella 5.1: Corrispondenza tra deployment locale e deployment su AWS

Fase 1 (locale)	Fase 2 (AWS)
Macchine virtuali Multipass	Istanze EC2
Pod PostgreSQL	Amazon RDS
Pod Redis	Amazon ElastiCache
Pod RabbitMQ	Amazon MQ (AMQPS, porta 5671)
Volume <code>emptyDir</code> per le immagini	Bucket Amazon S3
Docker Hub	Amazon ECR
Service NodePort 30080	NodePort dietro Network Load Balancer
Stato Terraform su file locale	Bucket S3 con lock su DynamoDB
Deployment manuale	GitHub Actions con OIDC

Il dato più significativo è ciò che *non* compare in tabella: il codice dei microservizi, che non ha subito alcuna modifica.

La distribuzione delle immagini segue lo stesso modello nelle due fasi: costruzione sulla macchina di sviluppo, pubblicazione su un registro, prelievo autonomo da parte dei nodi. Le differenze riguardano la natura del registro pubblico e non autenticato nella prima fase, privato e integrato con IAM nella seconda e la gestione delle credenziali, che su ECR richiedono un token temporaneo della durata di dodici ore, rigenerato a ogni distribuzione.

5.2 Problemi affrontati

Una parte rilevante del lavoro è consistita nella diagnosi di problemi non previsti. Se ne riportano i principali, con il procedimento seguito.

5.2.1 Concorrenza nella creazione dello schema

Al primo avvio dell'applicazione, con database vuoto, il servizio terminava con un errore di violazione di vincolo di unicità durante la creazione delle tabelle. La causa risiedeva nell'esecuzione concorrente: due worker del server applicativo, e in seguito due servizi distinti,

eseguiivano contemporaneamente la creazione dello schema sullo stesso database vuoto. Il primo completava l'operazione, il secondo falliva.

La soluzione adottata gestisce l'eccezione con un rollback e prosegue: chi arriva secondo trova lo schema già creato, condizione perfettamente accettabile.

5.2.2 Assenza di ordinamento all'avvio in Kubernetes

Il problema precedente si è ripresentato in forma diversa dopo il passaggio a Kubernetes. In Docker Compose la direttiva `depends_on` garantisce che i servizi applicativi vengano avviati soltanto dopo che i sistemi di persistenza rispondono. Kubernetes non prevede un meccanismo equivalente: i pod vengono avviati in parallelo e in ordine non deterministico.

Il servizio poteva quindi trovare il database non ancora disponibile. La gestione generica dell'eccezione introdotta in precedenza mascherava l'errore, lasciando il servizio in esecuzione ma privo di tabelle. La soluzione distingue le due condizioni: l'indisponibilità del database comporta un nuovo tentativo dopo alcuni secondi, la presenza di tabelle già create comporta la prosecuzione.

```
1 for _ in range(30):
2     try:
3         db.create_all()
4         break
5     except (IntegrityError, ProgrammingError):
6         db.session.rollback() # tabelle gia' create da un altro processo
7         break
8     except OperationalError:
9         db.session.rollback() # database non ancora pronto
10        time.sleep(2)
```

Listing 5.1: Attesa del database all'avvio

La lezione generale è che in un sistema distribuito la resilienza all'ordine di avvio deve risiedere nell'applicazione, non nell'orchestratore.

5.2.3 Schema condiviso tra servizi

Un terzo problema, emerso dallo stesso contesto, riguardava la tabella dei piatti, utilizzata in scrittura da menu-service e in sola lettura da order-service. Quest'ultimo ne dichiarava una versione ridotta, contenente i soli campi effettivamente utilizzati. Quando order-service veniva avviato per primo, creava una tabella incompleta, e le successive interrogazioni del menu fallivano per colonna inesistente.

La soluzione immediata è stata allineare le due dichiarazioni. La riflessione più generale è che la condivisione di tabelle tra microservizi costituisce un compromesso rispetto al modello puro, che prevede un database per servizio; è una scelta accettabile in un progetto di queste dimensioni, ma va riconosciuta come tale.

5.2.4 Vincoli dell'ambiente di sviluppo

La creazione delle macchine virtuali locali si è rivelata problematica. Multipass utilizzava VirtualBox come hypervisor, configurazione nella quale tutte le macchine ricevevano il medesimo indirizzo privato, non raggiungibile dall'esterno né utile alla comunicazione reciproca.

La diagnosi, condotta ispezionando le interfacce di rete all'interno di una macchina, ha portato a identificare la causa. Il passaggio a Hyper-V non era tuttavia percorribile, poiché l'edizione Home di Windows non lo include. La soluzione è consistita nello spostare l'intera

catena di strumenti in WSL2, che espone il supporto alla virtualizzazione: Multipass utilizza in tale ambiente il driver QEMU. Va peraltro osservato che Ansible non è eseguibile nativamente su Windows, per cui il passaggio sarebbe stato comunque necessario.

5.2.5 Non determinismo nella selezione dell'immagine

Le istanze EC2 sono definite utilizzando un *data source* che seleziona l'immagine Ubuntu più recente pubblicata da Canonical. Questa impostazione introduce però un elemento di non determinismo: alla pubblicazione di una nuova immagine, l'esecuzione successiva di `apply` rileva una differenza e procede alla sostituzione delle istanze.

Il fenomeno si è verificato durante lo sviluppo, comportando la perdita del cluster e la necessità di rieseguire il playbook. Il rimedio consiste nel fissare l'identificativo dell'immagine o nell'escludere tale attributo dal confronto tramite un blocco `lifecycle`.

5.2.6 Identificatori immutabili nel token OIDC

Il problema più insidioso ha riguardato l'autenticazione della pipeline. Il tentativo di assunzione del ruolo falliva con un errore di autorizzazione, pur essendo il ruolo esistente, il secret configurato e la condizione apparentemente corretta.

La diagnosi è stata condotta per esclusione. Ampliando temporaneamente la condizione a qualunque repository, l'autenticazione riusciva: il problema non risiedeva quindi né nell'identity provider né nell'audience, ma nel confronto sul nome del repository. Restringendo la condizione al solo proprietario, l'autenticazione tornava a fallire, circostanza in apparente contraddizione con il nome effettivo del repository.

Si è reso quindi necessario ispezionare il contenuto reale del token. Un passo aggiuntivo nel workflow ne ha decodificato la sezione centrale, rivelando il valore effettivo del campo `sub`:

```
1 repo:vitomarino02-del1@276350598/Cloud-Mensa-AWS-Version@1317345786:ref:refs/heads/main
```

Listing 5.2: Contenuto reale del claim

GitHub include ora, accanto ai nomi di proprietario e repository, i rispettivi identificatori numerici immutabili. La ragione è che i nomi possono essere modificati, gli identificatori no: l'inclusione di questi ultimi impedisce che una policy scritta per un repository continui a valere per un repository diverso che ne abbia ereditato il nome.

La condizione è stata quindi riscritta accettando entrambi i formati, mantenendo la restrizione al solo repository del progetto.

L'aspetto metodologicamente rilevante è che il problema non era diagnosticabile ispezionando la configurazione: la risposta era contenuta nel token, ed è stata ottenuta osservandone il contenuto effettivo anziché quello atteso.

5.2.7 Visibilità delle immagini su S3

Al termine della migrazione le foto dei piatti non venivano visualizzate: gli URI restituiti puntavano correttamente al bucket, ma questo era configurato senza accesso pubblico e restituiva un errore di autorizzazione.

Sono state valutate due soluzioni. La prima, adottata, consiste in una policy che consente la sola lettura degli oggetti; le chiavi sono identificatori casuali e la scrittura resta riservata al ruolo IAM dei nodi. La seconda, più restrittiva, avrebbe previsto la generazione di URL firmati a scadenza da parte del menu-service, mantenendo il bucket completamente privato: è la soluzione che si adotterebbe in un contesto reale con contenuti riservati.

5.3 Considerazioni sui costi

L'infrastruttura AWS comporta un costo dell'ordine di alcuni euro al giorno con tutti i componenti attivi. La possibilità di distruggere e ricreare l'intero ambiente tramite un singolo comando ha consentito di limitare la spesa all'effettivo tempo di utilizzo.

Questa possibilità è una diretta conseguenza dell'approccio Infrastructure as Code: l'infrastruttura, essendo interamente descritta in file di configurazione, diventa un artefatto riproducibile. Le uniche risorse mantenute attive sono il bucket dello stato e la tabella di lock, il cui costo è trascurabile.

5.4 Conclusioni

Il progetto ha realizzato un'applicazione a microservizi e ne ha verificato il deployment in due ambienti differenti, mantenendo invariato il codice applicativo. La proprietà è stata ottenuta esternalizzando integralmente la configurazione e introducendo un livello di astrazione sui componenti dipendenti dall'ambiente, come lo storage delle immagini.

L'infrastruttura è descritta come codice in entrambe le fasi, il che ne consente la ricreazione integrale e riproducibile. La configurazione dei nodi è affidata a un playbook idempotente riutilizzato in entrambi i contesti, a conferma dell'indipendenza degli strumenti di configuration management dall'infrastruttura sottostante.

La gestione delle credenziali è stata trattata come requisito e non come dettaglio: nessun segreto è presente nei repository, le password sono generate automaticamente e conservate cifrate, e l'autenticazione della pipeline avviene tramite credenziali temporanee.

I problemi incontrati, documentati nel presente capitolo, hanno costituito la parte più formativa del lavoro. In particolare, la diagnosi del problema sull'autenticazione OIDC ha richiesto di abbandonare l'ispezione della configurazione per osservare il dato effettivamente scambiato tra i sistemi: un metodo che si è rivelato risolutivo dove il ragionamento sulla configurazione attesa non lo era stato.